GPU Programming and Architectures

SS 2026

Project Report

Group 3

Name	Mat.Nummer
Sebastian Dengg	12114779
Konstantin Hauser	12040134
Johannes Polzer	12217047
Jonas Müller	12223225
Jonas Janser	11921706

June 23, 2026

1 Introduction

The goal of this project is the implementation of a high-performance, GPU-accelerated pipeline for the detection and classification of jigsaw puzzle pieces. Jigsaw puzzle pieces are characterized by four edges, which can be straight (L), a knob (V), or a hole (C).

Processing these images requires a transition from 2D image data to 1D geometric signals. By leveraging the parallel processing power of NVIDIA GPUs via CUDA, we aim to minimize execution time for tasks such as image filtering, connected-component labeling, and signal analysis.

2 Description of the Solution

Our solution follows a multi-stage pipeline, starting with raw RGB image data and ending with a visualized classification.

2.1 Architectural Overview

The system is divided into three main logical phases:

1. **Image Processing:** Preprocessing and filtering to isolate puzzle pieces and remove noise from the image.

2. **Boundary Extraction:** Using Connected Component Labeling (CCL) and Pointer Jumping to extract ordered and labeled contours.

3. **Signal Analysis and Visualization:** Converting contours into distance-based signals to identify corners and edge types. Overlay the labels onto the final output image.

2.2 Implementation Details

2.2.1 Preprocessing and Filtering

The preprocessing stage transforms the input image into a binary mask in which foreground pixels correspond to puzzle pieces.

Decoding The input image is first decoded using NVIDIA's `nvJPEG` library, eliminating the host-to-device transfer that a CPU-side decode would require. `nvjpegGetImageInfo` parses the JPEG header in advance to extract image dimensions and allocate pipeline buffers before decoding is performed. The output format `NVJPEG_OUTPUT_RGBI` produces interleaved RGB bytes (*R - G - B - R..*), matching the `uchar3` layout, expected by the following conversion kernel. The library provides an option to convert to single-channel grayscale directly with `NVJPEG_OUTPUT_Y`, but for generality we launch a custom conversion kernel. Timing is possibly improved with the libraries conversion functionality. The conversion to grayscale reduces the amount of data processed by the following kernels and simplifies thresholding.

Gaussian Blur To reduce noise in the image, a Gaussian blur is applied to the grayscale image. The blur smooths small local variations and makes the following thresholding step more robust. The 2D Gaussian kernel is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right). \tag{1}$$

Since G is separable, $G(x, y) = G(x) \cdot G(y)$, the 2D convolution reduces to two successive 1D passes

$$I'(x, y) = \sum_{j=-r}^{r} g(j) \sum_{i=-r}^{r} g(i)\, I(x + i,\, y + j), \tag{2}$$

where the normalized discrete 1D kernel is

$$g(k) = \frac{\exp(-k^2/2\sigma^2)}{\sum_{m=-r}^{r} \exp(-m^2/2\sigma^2)}, \qquad k \in [-r,\, r]. \tag{3}$$

The kernel is truncated at radius $r = \lceil 3\sigma \rceil$. With $\sigma = 2.0$ this yields $r = 6$ and a 13-tap kernel. The kernel coefficients are computed and normalized on the host, before they are uploaded once to GPU constant memory. From there they are broadcast to all threads.

On the GPU, both passes are fused into a single kernel. The input tile including halos is loaded once from global memory into shared memory, the horizontal convolution result is stored in a second shared-memory buffer, and the vertical pass reads from it directly.

Histogram The histogram kernel counts the number of pixels at each intensity level [0, 255], building a 256-bin frequency distribution over the Gaussian output. The histogram kernel counts pixel frequencies across 256 intensity bins. To reduce global atomic contention, each warp first accumulates into its own private copy of the histogram in shared memory. Once all pixels are processed, the per-warp copies are summed bin-by-bin in shared memory and the result is written to global memory with a single `atomicAdd` per bin per block. This helps the OTSU to find the optimal threshold.

OTSU's Method After computing the histogram, Otsu thresholding determines a global threshold by maximizing the separation between foreground and background intensity distributions. This avoids the need for a manually chosen fixed threshold and allows the implementation to adapt to different input images. The Otsu kernel launches a single block of 256 threads, one per intensity bin. Cumulative pixel counts $\omega(t)$ and cumulative intensity sums $\mu(t)$ are computed via a parallel inclusive prefix scan (Hillis–Steele, $\mathcal{O}(\log 256) = 8$ steps) entirely in shared memory. All 256 threads then evaluate the inter-class variance

$$\sigma_B^2(t) = \omega_0(t)\,\omega_1(t)\,\big(\mu_0(t) - \mu_1(t)\big)^2 \tag{4}$$

in parallel, followed by a shared-memory reduction to find the maximum. The thresholded image is then converted into a binary image: pixels with intensity above the threshold are set to 255 (foreground) and all others to 0 (background).

Finally, morphological operations are applied to clean up the mask. An erosion step removes small isolated noise pixels, while a following dilation step restores the approximate size of the detected objects. This corresponds to an opening operation and improves the quality of the connected components extracted later.

The output of this stage is therefore a binary image that contains the segmented puzzle pieces and is suitable for connected-component labeling.

2.2.2 Connected Component Labeling

To identify individual puzzle pieces from the binary mask, we implement the **Playne-Equivalence algorithm** as described in [1]. This algorithm is specifically optimized for GPU architectures as it replaces the heavy use of global memory atomics and branch divergence found in traditional CCL approaches with a more structured, three-phase approach:

1. **Local Blocked CCL:** The local CCL performs a full CCL, but only on a subregion of the image. Consequently the results from the local CCLs have to be merged in a later step.

a) **Warp-Optimized Neighbor Discovery:** Before initializing labels, each thread must know the binary values of its neighbors. To avoid redundant Global Memory reads, we utilize Warp Shuffles(`__shfl_up_sync`). Since the block width is 32 (matching the GPU warp size), a thread can retrieve the value of the pixel to its left directly from the register of the adjacent thread. This reduces memory latency and ensures that each pixel value is read from Global Memory only once.

b) **Initial Labeling and Precedence:** Each pixel is initially assigned a label based on its coordinates within the tile. We then apply a specific connectivity rule: if a pixel is connected to its neighbor to the Top or Left, it "inherits" that neighbor's label. In the Playne-Equivalence algorithm, the Top neighbor is given precedence over the Left neighbor. This establishes an initial labeling in Shared Memory, where the pixel labels form a chain of pointers, pointing towards the pixel with the smallest index within a connected region. This pixel is called the root of a component.

c) **Union-Find:** Once initial pointers are set, the threads perform a Path Compression. Every thread follows the chain of pointers in Shared Memory until it finds its root and changes its own label to the root-label. This step propagates the root labels, but does not manage to fully identify the connected regions yet.

d) **Reduction and The "Missing Link":** Due to the top-neighbor precedence in the initialization-step a component can still include clusters of different labels (The top label will propagate downwards even if the left label would be smaller - This is an issue specifically for "V"-Shapes). Playne handles this by isolating specific edge pixels and then merging the clusters together.

e) **Global Label Initialization:** After a final path compression step to ensure every pixel points to its resolved root, the indices within the subregion are translated into Global Pixel Indices. These are then written to Global Memory.

2. **Border Reduction:** Because the local phase only identifies connectivity within isolated tiles, another reduction step is required to merge components that span across tile boundaries. This is done using specialized kernels that process and connect the horizontal and vertical seams of the tiles.

3. **Label Propagation:** In the final phase, every pixel once again traverses the new label pointers until it reaches the ultimate root. This results in a global label map where every pixel belonging to the same puzzle piece is assigned the exact same unique ID, which corresponds to the index of the root.

2.2.3 Boundary Extraction and Contour Ordering

Once pieces are uniquely labeled, the system must then transform the 2D image into a 1D signal for classification. This is achieved by creating a clockwise-ordered list of boundary pixels per piece and assigning them values according to their distance from the puzzle pieces centre of gravity.

Parallel Successor Mapping: In the first step we identify boundary pixels by checking for foreground pixels with at least one direct background neighbor. These extracted boundaries are saved for later use. Then, for every boundary-pixel, we determine its successor by scanning its 8-neighborhood (Moore-neighbors) in a clockwise fashion. This step transforms the contour of each puzzle piece into a **circular linked list**. This connection is cut at the root pixel, forming a chain of pixel pointers following the boundary in a clockwise manner.

List Ranking via Pointer Jumping: To transform the linked list into an ordered sequence of boundary points, we initially used a Parallel Pointer Jumping algorithm, chosen for its $O(\log N)$ theoretical complexity. However, empirical testing demonstrated that the synchronization overhead of this approach outweighed the benefits for our specific dataset. Consequently, we transitioned to a more efficient iterative approach: point ranks are assigned by iteratively traversing the boundary, performed in parallel across independent contour segments. This iterative method is more efficient in our use case because the boundaries are only about 2,500 pixels long. With such a small N the iterative overhead is minimal. While Parallel Pointer Jumping is theoretically faster it would likely only outperform the iterative approach on much higher-resolution images where the boundary lengths are large enough to justify the algorithm's overhead.

The final output of this step is a list of all detected boundary pixels, where each pixel is assigned two data points:

- **Label:** Derived from the Playne-CCL and Border Reduction phases, this unique identifier groups pixels by their specific puzzle piece.
- **Rank:** The rank represents the pixel's exact sequential index along the clockwise contour of a specific puzzle piece.

Together they identify each boundary point uniquely.

2.2.4 Signal Generation

In order to classify the puzzle pieces a signal must be generated for each piece which represents the distance of a contour point to the center of mass of the piece.

In order to do that, the following steps are executed

- Compact boundary pixel ids
- Create and sort signal points
- Compute centroids from sorted points
- Filter out small components (noise)
- Compact valid centroids
- Apply centroids to points and remove noise

Compact boundary pixel ids To avoid idle threads, `thrust::make_counting_iterator` and `thrust::copy_if` are used to produce an array with only the indices of contour pixels.

Create and sort signal points Then structures are created for all contours pixels. These structures include the x-, y-coordinates, label, distance to centroid and rank. The distance is left as 0 at this point. Furthermore, this new array gets sorted by the key where

$$key = (\texttt{label} \ll 32) \ |\texttt{rank}.$$

Compute centroids from sorted points In order to calculate the center of mass of each puzzle piece, the points get transformed into tuples of `(x,y,1)`. With a sum reduction by the key `label`, the sum of the x-, y-components and the number of pixels per piece contour get obtained.

Filter out small components (noise) Since there is some noise, components with a small number of contour pixels are ignored. This is done the same way as the *Compact boundary pixel ids*.

Compact valid centroids This step copies the valid structures of *Compute centroids from sorted points* and calculates the centroid.

Apply centroids to points and remove noise Here the distance is calculated for each contour pixel to its center of mass and gets written to the results array.

2.2.5 Signal Smoothing

To mitigate noise a moving-average filter is applied to the distance signal. The operation is implemented in parallel for each pixel on the boundary:

Each GPU thread is mapped to a single point on the boundary, allowing the entire dataset to be processed in a single kernel. Since puzzle contours are closed loops, the smoothing window must "wrap around" the segment boundaries. We implement a circular convolution logic using the modulo operator. This ensures that samples at the beginning of a contour correctly incorporate neighbors from the tail of the sequence. By pre-calculating the `rank` (the local index within a specific piece) and the `segment_length` on the device, the kernel can dynamically calculate the base memory offset (`start = tid - rank`) for any given piece. This approach avoids the significant overhead of launching separate kernels for each individual puzzle piece.

The result is a refined distance signal $D'(i) = \frac{1}{2r+1} \sum_{j=-r}^{r} D(i+j)$ that preserves the fundamental geometry of the puzzle piece while suppressing jitter that could otherwise lead to false-positive corner detections.

2.2.6 Corner Detection

The identification of the corners of a puzzle piece is implemented as a two-stage algorithm: a parallel candidate generation phase on the GPU followed by a heuristic selection phase on the host, choosing the best candidates provided by the GPU stage.

1. Parallel Peak Detection (GPU)
The first stage identifies all potential corner candidates by analyzing the distance-to-centroid signal. We utilize a CUDA kernel to evaluate every boundary point in parallel. A point is classified as a candidate only if it satisfies two conditions:

1. **Local Maximality:** Within a configurable `peak_detection_radius`, the point must possess the strictly highest distance value. This is verified using circular indexing to maintain continuity across the contour's wrap-around point.

2. **Prominence Thresholding:** To filter out noise-induced micro-peaks, we calculate the prominence of the peak, defined as the difference between the point's distance D_i and the average distance of its neighbors within the peak-detection-radius R:
$$P = D_i - \frac{1}{2R} \sum_{j \in [-R,R], j \neq 0} D_{i+j}$$
Only points where $P \geq \tau_{prominence_threshhold}$ are stored as candidates.

2. Best Candidate Selection (Host)
While the GPU identifies all local features, a puzzle piece geometrically requires exactly four corners to define its boundary edges. The host-side function `select_corners_for_piece` performs a greedy selection to extract the four best candidates:

- **Scoring and Sorting:** Candidates are ranked by their prominence scores. This ensures that the sharpest, most "corner-like" features are prioritized regardless of their position in the memory array.

- **Distance Pruning:** A candidate is only accepted if its cyclic distance (the shortest path along the contour) to all previously selected corners exceeds a minimum threshold Δ_{min}

Once the top four candidates are selected, they are re-sorted by their `rank` (index along the contour).
By offloading the heavy neighborhood search to the GPU and reservedly using the CPU for the logical selection of exactly four points, the system achieves a robust balance between raw throughput and accuracy.

2.2.7 Classification

The classification stage assigns one of three edge classes to every side of a detected puzzle piece. The classes are:

- **L**: a straight edge,

- **V**: a protruding knob,

- **C**: an inward hole.

Using the four detected corners, the ordered contour is split into four edge segments. Each segment is analyzed relative to the straight line connecting its two corner points. If an edge is approximately straight, the contour points remain close to this line. If the edge contains a knob or a hole, the contour deviates significantly from the line.

For every edge, the deviation of the contour points from the baseline is evaluated. To avoid corner artifacts, the classification focuses mainly on the central part of the edge rather than the points directly next to the corners. The average signed deviation is then used to distinguish between the three classes. A small deviation indicates a straight edge. A strong deviation in one direction indicates a protruding knob, while a strong deviation in the opposite direction indicates a hole.

The result of the classification stage is a four-character label for each puzzle piece. Each character describes one edge of the piece in contour order. For example, a label such as `LVCC` describes a piece with one straight edge, one knob, and two holes.

2.2.8 Visualization

The visualization stage produces an RGB-image that overlays classification results directly onto the original RGB source. To maintain high performance we implement a hybrid rendering algorithm.

1. Pixel Rendering
Standard CPU based overlaying creates a bottleneck, where the original image would have to be transferred back to the CPU. We avoid this by utilizing a "Pixel Issue" architecture:

- **Buffering:** The host identifies and calculates the positions for visual elements (bounding boxes, contour points, and bitmap characters). Instead of modifying the image directly, the host generates a `thrust::host_vector` of `issued_pixel_t` objects. Each object contains a target pixel index and an RGB color.

- **Parallel Visualization:** Once the command buffer is populated, it is uploaded to the GPU. A specialized CUDA kernel is launched where every thread processes one "issued pixel." This allows thousands of visual annotations to be written into the global `uchar3` image buffer in parallel, achieving significantly higher throughput than serial CPU pixel-setting.

2. nvJPEG Hardware Compression
The final and most significant optimization in the visualization stage is the use of the `nvJPEG` library. Standard image-saving libraries (such as *stb_image_write*) operate on the CPU, requiring a costly synchronization and download of the entire raw image (often several megabytes) before performing slow, software-based compression.

By utilizing the `nvJPEG` encoder, the image data remains in GPU memory. The hardware-accelerated JPEG engine on the NVIDIA GPU performs the encoding in parallel. Only the final, significantly smaller compressed bitstream is downloaded to the host to be

written to disk. This reduces the total visualization latency by an order of magnitude compared to traditional CPU-based JPEG encoding.

2.3 Global Optimizations

A significant optimization was the reduction of repeated resource creation and memory allocation during batch processing. Since the program processes many images in a single execution, it is inefficient to allocate and destroy temporary GPU resources independently for every image. Therefore, several resources that are required repeatedly were moved outside the per-image processing function and are created only once at program startup.

2.3.1 Reusable nvJPEG Encoder and Decoder

The input images are decoded using NVIDIA's `nvJPEG` library, and the final annotated output images are encoded again as JPEG files. In the initial version, the required `nvJPEG` handles and states were created and destroyed for each processed image. This introduces unnecessary overhead, because handle creation is not part of the actual image-processing algorithm and does not depend on the image content.

To avoid this overhead, the decoder and encoder objects are initialized once before the image-processing loop starts. The same decoder state is reused for every input image, and the same encoder state is reused for every output image. Only after all images have been processed are the `nvJPEG` resources destroyed. This reduces repeated setup work and makes the runtime more stable when processing a large number of images.

Conceptually, the optimization changes the structure from

[create decoder/encoder → process image → destroy decoder/encoder]

for every image to

[create decoder/encoder once → process all images → destroy decoder/encoder once.]

This is especially beneficial in a batch-processing scenario, because the setup cost is paid only once instead of once per image.

2.3.2 Workspace-Based Device Memory Management

Another important optimization was the introduction of reusable GPU workspaces. Many stages of the pipeline require temporary device buffers. Examples include buffers for grayscale images, binary masks, segmentation results, connected-component labels, pointer-jumping data, centroid accumulation, sorting keys, smoothed contour signals, and visualization data.

In the initial implementation, many of these buffers were allocated and freed inside the individual pipeline stages. Repeated calls to `cudaMalloc` and `cudaFree` can be expensive, and `cudaFree` may also introduce synchronization. This is undesirable in a high-throughput pipeline where many images of the same or similar resolution are processed sequentially.

To reduce this overhead, persistent workspace structures were introduced. A workspace owns device buffers and keeps track of their current capacity. Before a pipeline stage uses a buffer, the workspace checks whether the existing allocation is large enough. If the buffer already has sufficient capacity, it is reused directly. A new allocation is only performed if the current image requires more memory than previously allocated.

The basic idea is

[allocate only if required capacity increases]

instead of

[allocate and free for every image.]

This reduces allocation overhead and improves runtime consistency across images. Since all images in the benchmark have the same resolution, most workspace buffers are allocated during the first image and reused for all following images.

3 Benchmarking Results

The following performance measurements were taken using the `GLOBAL_TIMER` macro in the CUDA code, and via the normal `time` library in python. The benchmarks ran on the same machine using an AMD EPYC 7313 processor and an NVIDIA Tesla T4 GPU.

The reference values in python come from the provided example code, extended by a simple python implementation of the same classification and visualization algorithms used in the CUDA implementation, just not parallelized nor optimized.

Looking at those results, the speedup is about 4 orders of magnitude. It depends on the number of puzzle pieces in the image and its resolution, since the parallelization of the CUDA code scales primarily with the resolution, but there are also some parts which scale with the number of puzzle pieces.

Preprocessing takes by far the biggest part of the time, in the parallelized version about half of the complete runtime. About 86% of that is the transfer of the image data to the device memory and decoding the JPEG compression.

To solve this bottleneck, we use `cudaStream`. If there are multiple images processed, this enables the data transfer and JPEG decoding to run in parallel with the processing and analyzing of the previous image. Running with only one stream, processing all the images (20 images, 1998 puzzle pieces) takes 3.8s. Running on 4 streams it takes only 2.2s because of the parallelization. With some luck, we get runs with under 2.1s.

The runtime tables provided are executed with only one stream to get sane outputs, since multiple streams sometimes have to wait for each other, therefore increasing the time each step takes. Because of the parallelization of multiple images, the best speed can be achieved when processing many images at a time. This is also the reason why the processing time of each image multiplied by its runtime is more than the actual time it took to process all the images.

Table 1: Execution time for different pipeline stages
(Image `18_p1.jpg`, 66 pieces, Resolution: 5100×7016 px)

Stage	**Python baseline Time [s]**	**GPU accelerated Time [ms]**
Preprocessing	150.06	62.373
CCL & Pointer Jumping	701.18	14.314
Signal Generation	–	10.818
Corner Detection	–	0.466
Classification	9.384	0.313
Visualization	23.26	10.028
Total Execution	**884.45** s	**99.701** ms

Table 2: Execution time for different pipeline stages
(Image `18_p2.jpg`, 34 pieces, Resolution: 5100 × 7016 px)

Stage	**Python baseline Time [s]**	**GPU accelerated Time [ms]**
Preprocessing	140.99	41.985
CCL & Pointer Jumping	677.05	13.998
Signal Generation	–	1.962
Corner Detection	–	0.298
Classification	6.353	0.168
Visualization	20.27	16.219
Total Execution	**844.67** s	**76.106** ms

4 Problems

There are some misclassifications in the given dataset. One is the fact, that the lines above the numbers written on the puzzle pieces are sometimes drawn up to the edges of the puzzle pieces. This confuses the boundary detection and the boundary follows the line "into" the piece and back out. This sometimes gets misclassified as a corner, therefore the rest of the classification is also wrong.

The second problem comes from dirt directly on the edges of the puzzle pieces. Our algorithms can deal with almost all the dirt, but there are some pieces where some pieces of dirt get recognized as corners, breaking the rest of the classification.

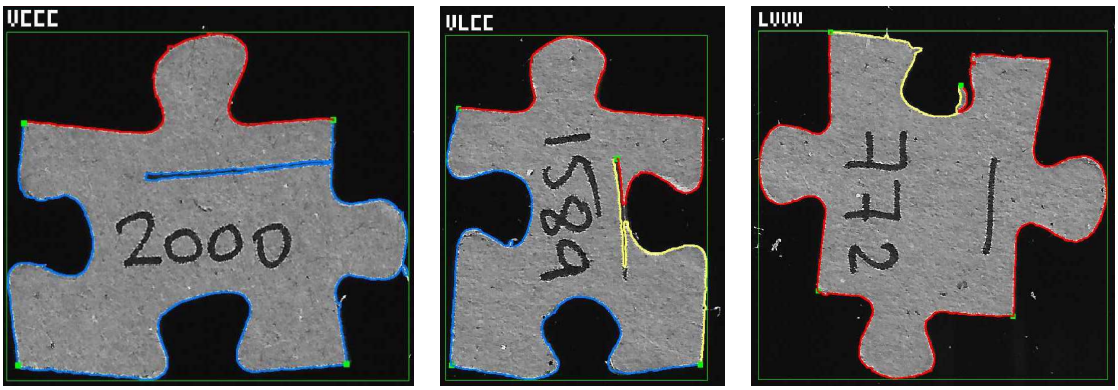


Figure 1: Classification problems

5 Possible Improvements

While the current implementation is functional and performant, several improvements could be made to improve classification quality:

- **Adaptive Thresholding:** Although Otsu thresholding is used to automatically determine a global threshold, the method can still fail under strong illumination gradients. A possible improvement would be adaptive or locally computed thresholding, which could make the segmentation more robust to uneven lighting.

- **Dynamic classification parameters:** Currently, the classification parameters are hardcoded to work with the provided image resolution and puzzle piece sizes. These parameters should be calculated from the images resolution and the size of the puzzle pieces. The classification might also be implemented as some kind of machine learning model.

- **Better images:** In the given images, there are some pieces with considerable amounts of dirt on the pieces edges. As mentioned, this leads to some wrong classifications. The lines written above the numbers on the puzzle pieces are sometimes drawn over the edges, also leading to misclassifications.

References

[1] D. P. Playne and K. Hawick, "A new algorithm for parallel connected-component labelling on gpus," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 6, pp. 1217–1230, 2018. DOI: 10.1109/TPDS.2018.2799216.